

Módulo 4 · Conocimientos en Git

Comandos solicitados y justificación de su uso (git cherry-pick).

1. Escenario planteado

Estás en la rama `feature/new-login`. Un compañero fusionó en `main` un commit que corrige un **bug crítico de seguridad** (hotfix). Necesitas incorporar **únicamente ese commit** a tu rama, sin arrastrar el resto de cambios de `main`.

2. Comando y estrategia: git cherry-pick

```
# 1. Estado limpio antes de operar
git status

# 2. Actualizar referencias del remoto
git fetch origin main

# 3. Identificar el hash del commit de seguridad en main
git log origin/main --oneline -n 5
# Ejemplo: alb2c3d "fix(security): sanitize auth input token to prevent injection"

# 4. En feature/new-login, aplicar SOLO ese commit
git cherry-pick alb2c3d

# 5. Si hay conflicto: resolver, marcar y continuar
# git status
# git add <archivos-resueltos>
# git cherry-pick --continue

# 6. Verificar el commit incorporado
git log -1 --stat

# 7. (Opcional) cancelar la operación si falla
# git cherry-pick --abort
```

3. Justificación de la decisión

`git cherry-pick <hash>` extrae un commit individual por su SHA-1 y lo reaplica sobre la rama actual mediante un **3-way merge** del parche. Es la única opción que trae exactamente un commit.

Por qué NO git merge main

- Un merge traería **todos** los commits acumulados de `main` (features incompletas de otros desarrolladores, cambios que desestabilizan la rama).
- Violaría el requisito de aislar el parche de seguridad.

Por qué NO git rebase main

- Un rebase **reescribe el historial** de `feature/new-login` y lo coloca sobre la punta de `main`, trayendo la totalidad de sus cambios.

- Además modifica los SHAs de los commits locales (complicación y riesgo en trabajo compartido).

Conclusión

`cherry-pick` minimiza el alcance del cambio (cirugía quirúrgica), mantiene el historial intacto y es la práctica estándar para backport de hotfixes.

4. Complemento: modelo de branching recomendado (GitFlow adaptado)

Rama	Rol	Reglas
main	Producción auditada	Protegida; solo merges de releases/hotfixes
develop	Integración continua	Despliega a QA; fuente de las features
feature/*	Historias cortas (1-3 días)	Nacen de develop; se fusionan con PR
hotfix/*	Emergencias P1 desde main	Merge a main con tag semántico y backport a develop

4.1 Protocolo de hotfix en producción

```
# 1. Crear la rama de emergencia desde main
git checkout main && git pull origin main && git checkout -b hotfix/calculo-prima-colectiva-v1.0.1

# 2. Corregir + test de regresión (mvn test) + commit

# 3. Merge a main con tag semántico y despliegue
git checkout main && git merge --no-ff hotfix/calculo-prima-colectiva-v1.0.1 \
    && git tag -a v1.0.1 -m "HOTFIX: correccion calculo prima" && git push origin main --tags

# 4. Backport a develop y limpieza
git checkout develop && git merge --no-ff hotfix/calculo-prima-colectiva-v1.0.1 \
    && git push origin develop && git branch -d hotfix/calculo-prima-colectiva-v1.0.1
```

4.2 Buenas prácticas de equipo

- **Conventional Commits:** mensajes estándar — `feat(polizas): calculo de prima con IPC`, `fix(riesgos): bloqueo adicion en poliza individual`.
- **Branch protection:** push directo bloqueado a `main` / `develop`, mínimo 1 aprobación y CI en verde.
- **Squash & merge** en PRs: historial de `develop` limpio y atómico.

Fuente: *spring-boot-backend/README.md* (Módulo 4) y pestaña BBDD & Git (*src/components/DatabaseAndGitView.tsx*) de la plataforma.